Pembahasan Singkat KSN 2020 Bidang Informatika

Daring, 16 Oktober 2020

Soal KSN 2020 Bidang Informatika

Kode	Judul Soal	Pembuat Soal	Disiapkan oleh
1A	Pertahanan Negara	Abdul Malik Nurrokhman	Reynaldo Wijaya
1B	Tumpukan Balok	Rico Filberto	Rico Filberto
1C	Menjaga Jarak	Abdul Malik Nurrokhman	Abdul Malik Nurrokhman
2A	Makhluk Jahat	Prabowo Djonatan	Hocky Yudhiono
2B	Pilar Ajaib	Andreas Martin, Lie Maximilianus Maria Kolbe	Lie Maximilianus Maria Kolbe
2C	Mencari Bola	Prabowo Djonatan	Prabowo Djonatan

Pertahanan Negara

- Diberikan wilayah negara berbentuk persegi panjang N x M dan disuruh membagi menjadi K wilayah pertahanan
- Masing-masing wilayah berbentuk persegi panjang yang sisinya sejajar dengan sisi wilayah negara
- Antar wilayah pertahanan tidak boleh beririsan namun sisinya boleh berhimpit
- Luas masing-masing wilayah diusahakan tepat A[i]



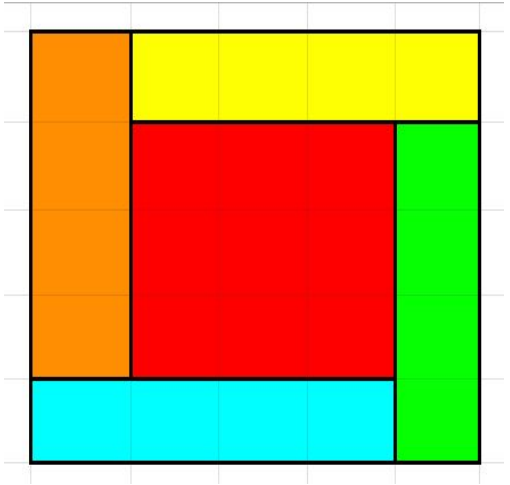
SC 1 - 0 Netizen

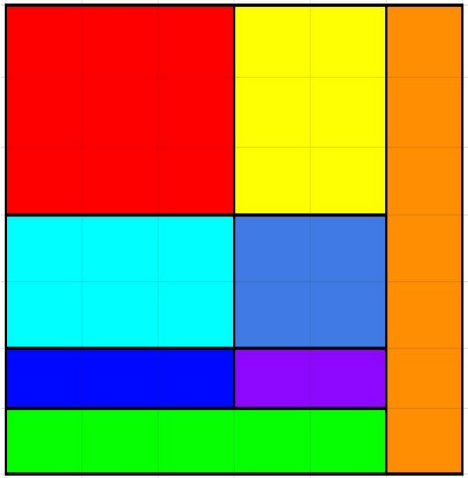
Kasus Uji 1 sampai 3

- Kasus uji 1 : A = {4,4,4,4,9}
  - Wilayah 5 dipasang di tengah berukuran 3 x 3
  - Sisanya mengisi bagian sisi-sisinya dengan ukuran 1 x 4
- Kasus uji 2 : A = {2,3,4,5,6,6,7,9}
  - Wilayah 3 berukuran 2 x 2
  - Wilayah 5 dan 6 berukuran 2 x 3
  - Wilayah 8 berukuran 3 x 3
  - Terdapat sangat banyak kemungkinan
  - Bisa dicoba manual di kertas
- Kasus uji 3 : N = 1
  - Wilayah i berukuran 1 x A[i]
  - Dipasang memanjang menjadi satu baris

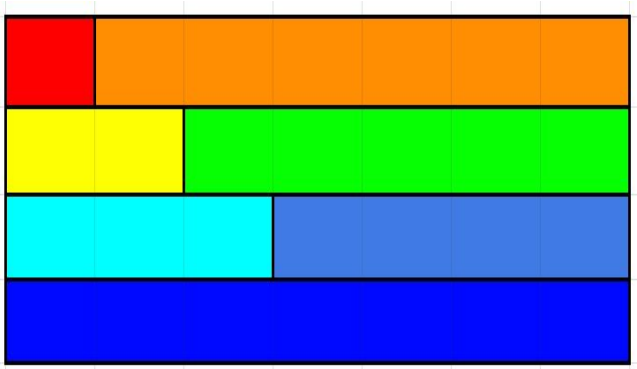
Kasus Uji 1, 2 dan 3

- Kasus uji 1 : A = {4,4,4,4,9}
  - Wilayah 5 dipasang di tengah berukuran 3 x 3
  - Sisanya mengisi bagian sisi-sisinya dengan ukuran 1 x 4
- Kasus uji 2 : A = {2,3,4,5,6,6,7,9}
  - Wilayah 3 berukuran 2 x 2
  - Wilayah 5 dan 6 berukuran 2 x 3
  - Wilayah 8 berukuran 3 x 3
  - Terdapat sangat banyak kemungkinan
  - Bisa dicoba manual di kertas
- Kasus uji 3 : N = 1
  - Wilayah i berukuran 1 x A[i]
  - Dipasang memanjang menjadi satu baris





Kasus Uji 4



- Kasus uji 4 : A[i] = i
  - N x M
  - = 99999 x 50000
  - 1 + 2 + 3 + ... + K-1 + K
  - = (1 + K-1) + (2 + K-2) + (3 + K-3) + ... + ((K-1)/2 + (K+1)/2) + K
  - = K x (K+1)/2
  - = 99999 x 50000
  - N = K = 99999
  - M = (K+1)/2 = 50000
  - Wilayah i dipasang dengan ukuran 1 x i
  - Setiap kolom i selain kolom terakhir ditempati wilayah i dan K - i
  - Kolom terakhir ditempati wilayah K
  - Contoh untuk 7 x 4

Kasus Uji 5

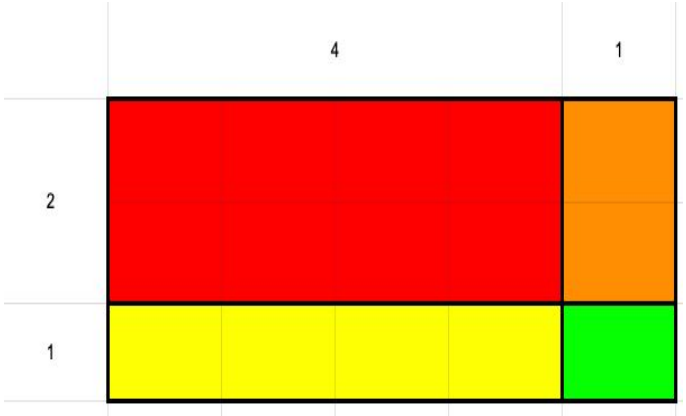
- Kasus uji 5 : A[i] = konstan
  - N x M = K x A[i]
  - PQ x RS = PR x QS
  - Wilayah negara bisa dibentuk grid P x R, masing-masing berukuran Q x S
  - Masing-masing wilayah pertahanan dipasang di salah satu grid
  - Namun di kasus uji ini nilai P, Q, R, dan S sudah diberikan di bagian subsoal
  - contoh N = 2 x 5, M = 3 x 7, K = 2 x 7, A_i = 3 x 5

Kasus Uji 6 dan 7

- Kasus uji 6 : N = 2, K = 20
  - Kemungkinan untuk A[i] ganjil :
    - Berukuran 1 x A[i] dan dipasang di baris pertama
    - Berukuran 1 x A[i] dan dipasang di baris kedua
  - Kemungkinan untuk A[i] genap :
    - Sama seperti 2 kemungkinan yang ada di kasus A[i] ganjil
    - Berukuran 2 x A[i] dan dipasang memenuhi kedua baris
  - Brute force semua 2^{banyak ganjil} x 3^{banyak genap} kemungkinan
  - Cari kombinasi yang {total yang diisi di baris 1} = {total yang diisi di baris 2} = M/2
  - Pasang semua yang berukuran 1 x A[i] dari ujung kiri pada masing-masing baris
  - Pasang semua yang berukuran 2 x A[i] dari ujung kanan
- Kasus uji 7 : N = 2, M = 10000, K = 100
  - Seperti cara pada kasus uji 6 namun menggunakan dp knapsack

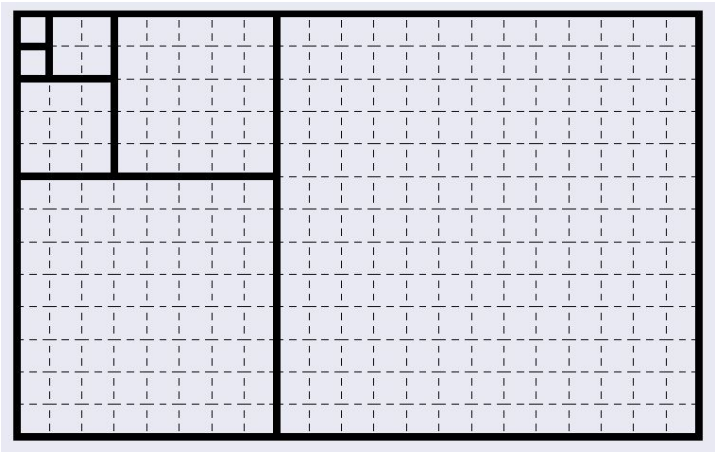
Kasus Uji 8

- Kasus uji 8 : A[i] = 2^(i-1)
  - Perhatikan setiap bit yang nyala pada N dan M
  - Buat grid dengan pembagian baris dan kolom berdasarkan bit yang nyala tersebut
  - Maka semua nilai 2^x untuk x dari 0 hingga K-1 ada di salah satu sel di grid tersebut
  - Pasang wilayah pertahanan di sel yang ukurannya sesuai
  - Contoh: N = 5, M = 3, K = 4, A = {1, 2, 4, 8}
  - N = 101, M = 011



Kasus Uji 9

- Kasus uji 9 : √A[i] = √A[i-1] + √A[i-2]
  - Dari rumus rekursi di atas dapat diperoleh A[i] = F[i]^2 dengan F[i] adalah barisan fibonacci
  - Sehingga wilayah i berukuran F[i] x F[i] dan berbentuk persegi
  - Pasang persegi-persegi tersebut selang-seling urut dari yang terkecil di posisi :
    - Menempel pada ujung kiri, pasang di baris paling atas
    - Menempel pada ujung atas, pasang di kolom paling kiri



Kasus Uji 10

- Kasus uji 10 : A[i] <= 100
  - Semua wilayah pertahanan berukuran 1 x A[i]
  - Pasang masing-masing wilayah berurutan pada setiap baris dari kiri ke kanan
  - Gunakan nilai A[i] yang paling banyak (modus)
  - Jika tidak cukup untuk dipasang di baris sekarang, maka cari nilai A[i] yang pas untuk memenuhi baris tersebut
  - Kemungkinan semua wilayah pertahanan terpasang sangat besar
  - Jika ada yang tidak terpasang, bisa diatasi dengan beberapa hal berikut :
    - diedit secara manual
    - cari 2 buah nilai A[i] yang jumlahnya pas untuk memenuhi baris tersebut
    - gunakan dp knapsack untuk nilai A[i] yang tersisa

Tumpukan Balok



- Diberikan N tumpukan balok, tumpukan balok ke i memiliki H[i] balok.
- Akan dilakukan operasi pengambilan balok dengan proses sebagai berikut:
  a. Berjalan menuju ke tumpukan dengan nomor terkecil yang terdapat balok,
  b. Misal saat ini berada di tumpukan ke-x, ambil 1 balok dari tumpukan tersebut. Lalu H[x] berkurang 1.
  c. Jika pada tumpukan ke-x+1 masih terdapat balok maka update nilai x menjadi x+1, lalu balik ke proses b
  d. Jika tidak balik ke proses a apabila masih terdapat tumpukan yang memiliki balok, selain itu proses pengambilan balok sudah selesai.
- Diberikan Q buah pertanyaan, misal pertanyaan ke-i adalah Ki, outputkan nomor tumpukan dari balok yang diambil pada urutan ke-Ki.

Solusi Subsoal 1-4

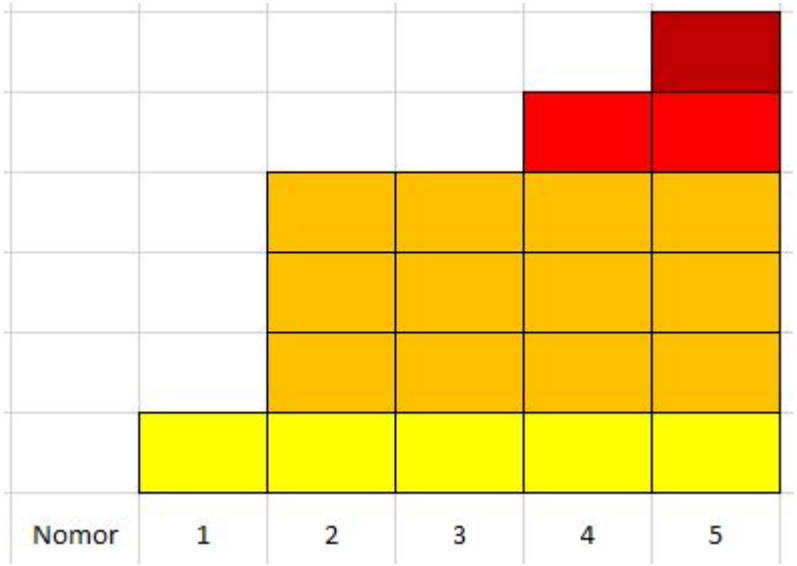
- Subsoal 1 dan 2
  - Kalian hanya perlu simulasikan proses pengambilan menggunakan excel (atau di kertas juga bisa).
- Suboal 3
  - Karena setiap tumpukan memiliki jumlah balok yang sama, maka urutan pengambilan balok yang terjadi akan selalu berbentuk 1, 2, 3, ..., N, 1, 2, 3, ..., N, ..., 1, 2, 3, N.
  - Sehingga untuk setiap pertanyaan anda hanya perlu outputkan (Ki - 1) % N + 1.
- Subsoal 4
  - Anda hanya perlu membuat program untuk mensimulasikan proses pengambilan seperti pada soal. (Tinggal menggunakan if dan for sudah cukup)

Subsoal 5 (H[i] <= 5)

- Subsoal 5
  - Karena H[i] <= 5, maka jumlah balok maksimal yang dapat diberikan pada input hanya 5*2*10^5 atau 10^6, sehingga untuk setiap pengambilan balok anda bisa mencatat jawaban nya di array berukuran 10^6.
  - Cara melakukan pengisian array jawaban juga sama seperti pada subsoal 4, tetapi kalian harus bisa mencari tumpukan dengan nomor terkecil yang terdapat balok dengan cepat (tidak melakukan linear search secara menyuluruh untuk setiap proses a).

Subsoal 6 (H[i] non-decreasing)

- Misalkan array tumpukan balok yang diberikan adalah (1, 4, 4, 5, 6). Maka kita bisa memecah tumpukan tersebut menjadi beberapa persegi panjang menjadi seperti berikut :
-



Subsoal 6 (H[i] non-decreasing)

- Perhatikan bahwa Pak Dengklek tidak bisa mengambil balok-balok pada persegi panjang berwarna jingga sebelum semua balok pada persegi panjang berwarna merah sudah habis diambil Pak Dengklek. Begitu juga balok pada persegi panjang berwarna merah tidak bisa diambil sebelum semua balok berwarna jingga habis diambil Pak Dengklek.
- Perhatikan juga bahwa setiap persegi panjang terdiri atas segmen(kiri, kanan) dan punya tinggi nya masing-masing. Sehingga kita dapat menghitung jumlah balok yang berada pada persegi panjang tersebut dengan mudah.
- Setelah itu kita dapat melakukan prefix sum untuk jumlah balok di masing-masing persegi panjang sehingga kita dapat melakukan binary search untuk menentukan pengambilan balok ke-Ki akan dilakukan pada persegi panjang yang mana.

Subsoal 6 (H[i] non-decreasing)

- Setelah mengetahui pengambilan dilakukan pada persegi panjang yang mana, untuk menetukan nomor tumpukan jawaban, kita hanya perlu menghitung hasil modulo sesuai dengan pengerjaan pada subsoal 3.
- Untuk menentukan batas (kiri, kanan) dan tinggi masing-masing persegi panjang anda cukup melakukan looping dari 1..N sambil mengecek selisih tinggi antara H[i-1] dengan H[i]. Hal ini dapat dilakukan karena nilai H[i] dijamin non-decreasing.

Subsoal 7 (Q = 1)

- Misalkan array tumpukan balok yang diberikan adalah (3, 4, 3, 2, 3, 4, 3). Klta juga dapat membetuknya menjadi beberapa persegi panjang seperti berikut:
-



Subsoal 7 (Q = 1)

- Alasan mengapa persegi berwarna kuning yang atau segmen(1, 7) hanya memiliki ketinggian 2 karena pada tumpukan nomor 4 hanya terdapat 2 balok. Lalu setelah di segmen(1, 7) dilakukan pengambilan sebanyak 2 kali maka segmen(1, 7) akan terpecah menjadi segmen(1, 3) dan segmen(5, 7).
- Karena menurut syarat proses pengambilan maka pengambilan pada segmen(5, 7) baru akan mulai dilakukan ketika semua balok pada segmen(1, 3) sudah habis diambil. Dengan demikian setiap saat suatu segmen dipecah menjadi 2 bagian maka kita dapat memperhitungkan pengambilan balok ke-Ki akan dilakukan pada segmen(1, 3) atau segmen(4, 7) dengan menghitung sisa balok yang ada di segmen(1, 3).

Subsoal 7 (Q = 1)

- Untuk mengitung sisa balok pada suatu segmen(le, ri) dengan cepat anda bisa menggunakan prefix sum dari jumlah balok pada masing-masing tumpukan. (prefix_sum[i] = H[1] + H[2] + ... + H[i])
- Untuk menentukan ketinggian suatu segmen dan juga index suatu segmen akan dipecah anda dapat menggunakan sorting dari kecil ke besar berdasarkan jumlah balok pada suatu tumpukan dan juga nomor tumpukan tersebut. (sortnya berarti array of pair {H[i], i})
- Dengan begitu untuk menjawab 1 pertanyaan dapat dilakukan dalam kompleksitas O(N) .

Subsoal 8 (N <= 5000)

- Kalian dapat menghitung untuk setiap pengambilan balok dari suatu segmen(left, right) akan dilakukan berapa kali. Misal min(left, right) menyatakan tumpukan balok dengan jumlah balok paling sedikit di antara semua balok dari nomor left hingga right, maka min(left,right) dapat digunakan untuk menghitung pengambilan balok dari suatu segmen(left, right) akan dilakukan berapa kali.
- Selanjutnya misalkan pivot = min_idx(left, right) merupakan nomor tumpukan di segmen(left, right) dimana H[pivot] = min(left, right). Maka setelah dilakukan pengambilan balok di range (left, right) sebanyak min(left, right) kali, segmen(left, right) akan terpecah menjadi 2 bagian segmen yaitu segmen(left, pivot-1) dan segmen(pivot+1, right).

Subsoal 8 (N <= 5000)

- Karena urutan pengambilan balok sangat penting untuk diperhatikan maka sebelum bisa mengambil balok pada segmen(pivot+1, right), anda harus memastikan semua tumpukan dengan nomor di antara (left, pivot-1) sudah memiliki 0 balok, alasan hal ini terjadi karena dari segmen(left, pivot+1) tidak terhubung dengan segmen(pivot+1, right) disebabkan H[pivot] sudah bernilai 0.
- Setelah anda berhasil menghitung apa yang ingin dihitung pada point pertama, maka anda juga dapat menentukan urutan segmen yang akan digunakan dalam proses pengambilan balok menggunakan point ke-3. Selanjutnya karena anda dapat mengetahui urutan penggunaan segmen dan juga untuk setiap segmen akan ada berapa banyak balok yang diambil, maka anda juga bisa menentukan suatu segmen akan mulai digunakan pada pengambilan balok ke-berapa dengan bantuan prefix sum.

Subsoal 8 (N <= 5000)

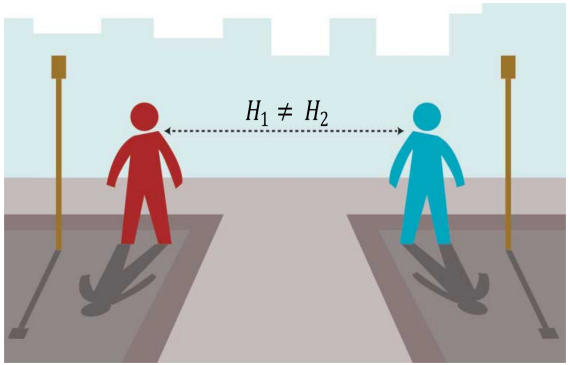
- Setelah anda berhasil menghitung itu semua, maka untuk setiap pertanyaan yang diberikan anda dapat melakukan binary search untuk menentukan pengambilan ke-Ki itu dilakukan di segmen yang mana selanjutnya untuk menentukan posisi nomor tumpukannya, anda hanya perlu melakukan modulo sederhana.
- Untuk bisa menghitung min_idx(left, right) dan min(left, right) anda cukup menggunakan linear search.
- Maka kompleksitas untuk pre-compute semua segmen adalah O(N^2) dan kompleksitas untuk menjawab suatu pertanyaan adalah O(log N).

Subsoal 9 (Full)

- Cara pengerjaan sama dengan subsoal 8 namun kita harus bisa mencari min_idx(left, right) dan min(left, right) dengan cepat bila tidak ingin mendapatkan verdict TLE.
- Salah satu cara-nya kita dapat menggunakan UFDS (Union Find Disjoint Set)
- Pertama-tama kita perlu melakukan sorting dari besar ke kecil berdasarkan jumlah balok pada suatu tumpukan dan juga nomor tumpukan tersebut. (sortnya berarti array of pair {H[i], i})
- Dengan menggunakan sort seperti itu kita dapat membentuk segmen-nya secara terbalik (menggabung-gabungkan segmen). Maka dari itu UFDS berguna untuk mencari batas kanan dan batas kiri dari suatu segmen dengan cepat.

Menjaga Jarak

- Terdapat N orang, masing-masing mempunyai tingkat kesehatan H[i]
- Cari minimum kelompok dengan aturan sebagai berikut :
  - Beberapa orang pertama masuk ke dalam kelompok pertama, beberapa orang selanjutnya masuk ke dalam kelompok kedua, dan seterusnya hingga beberapa orang terakhir masuk ke dalam kelompok terakhir
  - Dalam satu kelompok tidak boleh ada lebih dari M orang dengan tingkat kesehatan yang sama
  - Boleh mengeluarkan hingga K orang sehingga tidak masuk ke dalam kelompok manapun



Solusi Subsoal 3 dan 4

- Subsoal 3 : H[i] = 1
  - Menggunakan rumus (N - K) / M dibulatkan ke atas
- Subsoal 4 : N <= 2000, K = 0
  - Untuk setiap i cari nilai reach[i][0] sehingga dari orang ke-reach[i][0] (eksklusif) hingga orang ke-i (inklusif) tidak ada lebih dari M orang dengan tingkat kesehatan yang sama dan nilai reach[i][0] seminimum mungkin
  - Cari dp[i][0] yaitu banyak kelompok minimum yang diperlukan untuk memasukkan orang pertama hingga orang ke-i tanpa mengeluarkan siapa pun
  - dp[0][0] = 0
  - dp[i][0] = dp[reach[i][0]][0] + 1

Solusi Subsoal 5

- Subsoal 5 : N <= 2000, K <= 20
  - Cari reach[i][j] untuk setiap i dan j sehingga jika boleh mengeluarkan hingga j orang maka dari orang ke-reach[i][j] (eksklusif) hingga orang ke-i (inklusif) bisa tidak ada lebih dari M orang dengan tingkat kesehatan yang sama dan nilai reach[i][j] seminimum mungkin
  - Dalam implementasinya, banyak orang yang harus dikeluarkan minimum adalah total dari max(0,cnt[i]-M) untuk setiap cnt[i] = banyak orang dengan tingkat kesehatan i
  - Selanjutnya, cari dp[i][j] yaitu banyak kelompok minimum yang diperlukan untuk memasukkan orang pertama hingga orang ke-i dengan diperbolehkan mengeluarkan hingga j orang
  - dp[i][j] = 0, jika i <= j
  - dp[i][j] = min(dp[reach[i][k]][j-k] + 1) untuk semua 0 <= k <= j

Solusi Subsoal 6 dan 7

- Subsoal 6 : K = 0
  - Terdapat sebuah observasi bahwa reach[i][0] <= reach[i+1][0]
  - Sehingga semua reach[i][0] dapat dicari menggunakan sliding window aka two pointer
  - Cari dp[i][0] menggunakan cara seperti subsoal 4
- Subsoal 7 : K <= 20
  - Observasi yang ada pada subsoal 6 juga berlaku untuk semua j, reach[i][j] <= reach[i+1][j]
  - Cari reach[i][j] menggunakan sliding window dan dp[i][j] menggunakan cara seperti subsoal 5
- Subsoal 8 : tidak ada batasan khusus
  - Dari dp[i][j] yang dibuat di subsoal 5, dibuat backtrack[i][j] yaitu nilai k terkecil untuk mendapatkan nilai dp[i][j]
  - Ada yang kepikiran ? Tidak kan

Solusi Full

- Perhatikan bahwa setiap orang hanya terdapat 3 kemungkinan berikut :
  - Masih bisa untuk ikut kelompok yang sama dengan orang yang ada di depannya, sehingga
    - dp[i][j] = dp[i-1][j]
    - backtrack[i][j] = backtrack[i-1][j]
  - Dengan berat hati Pak Dengklek harus membuang orang ini, sehingga
    - dp[i][j] = dp[i-1][j-1]
    - backtrack[i][j] = backtrack[i-1][j-1] + 1
  - Membuat kelompok baru yang berbeda dengan kelompok yang ada di depannya, sehingga
    - dp[i][j] = dp[i-1][j] + 1
    - backtrack[i][j] = 0
- Sehingga kita hanya perlu 3 transisi tersebut tanpa mencoba semua nilai k
- Jangan lupa bahwa base case masih sama yaitu dp[i][j] = 0 untuk i <= j

Makhluk Jahat



- Diberikan matriks berukuran K × K yang menunjukkan hubungan kemenangan jenis elemen sihir.
- Diberikan 4 elemen sihir awal yang berbeda.
- Diberikan N ruangan, setiap ruangan jenisnya dapat berupa:
  - Jenis bertarung, Pak Dengklek akan menang bila setidaknya salah satu dari 4 elemen sihir yang dikuasainya menang melawan makhluk disana.
  - Jenis perpustakaan, Pak Dengklek dapat “menukar” elemen yang dimilikinya.

Solusi Subsoal 1

- Subsoal 1 (5 Poin) : Open Subtask dengan matriks kemenangan Unik
  - Perhatikan bahwa makhluk jahat yang **sudah muncul tidak muncul kembali**, dan untuk setiap elemennya, hanya ada satu elemen lain yang dapat mengalahkan elemen tersebut.
  - Cobalah untuk menukar elemen yang “sudah lewat” musuhnya dengan elemen yang “akan segera muncul” musuhnya.

Solusi Subsoal 2

- Subsoal 2 (9 Poin) : Open Subtask
  - Pisahkan elemen awal (1, 2, 5, dan 7) dan lakukan observasi makhluk dengan jenis apa saja yang dapat dilawan.

```
Elemen 1       : 00100000
Elemen 2       : 00010101
Elemen 5       : 11000100
Elemen 7       : 10001000
Bisa melawan   : 11111101
```

  - Perhatikan bahwa dengan mengganti elemen 7 dengan elemen 4 (01001110) Pak Dengklek dapat melawan semua musuh apapun jenisnya. Sehingga, semua ruang bertarung setelah perpustakaan pertama pasti akan dimenangi Pak Dengklek.

Solusi Subsoal 3

- Subsoal 3 (8 Poin) : Tidak ada perpustakaan
  - Perhatikan bahwa urutan perlawanan tidak akan berpengaruh, dapat dihitung untuk semua makhluk jahat, ada berapa yang memiliki elemen jenis ke-i.
  - Misalkan *array* **`hitung[i]`** sebagai banyaknya makhluk jahat dengan elemen ke-i, maka dapat dilakukan iterasi untuk setiap jenis i dari 1 hingga K.
  - Bila Pak Dengklek dapat mengalahkan monster dengan jenis ke-i, maka jawaban akhir akan ditambahkan sebanyak **`hitung[i]`**

Solusi Subsoal 4 dan 5

- Subsoal 4 (11 Poin) : Jumlah perpustakaan ≤ 3
  - Lakukan DFS, seperti pada graph. Karena jumlah perpustakaan ≤ 3, maka transisi untuk skill tidak akan banyak.
- Subsoal 5 (10 Poin) : Jenis elemen ada 5
  - Definisikan sesi bertarung sebagai segmen yang dipisahkan oleh dua buah perpustakaan. Untuk setiap segmennya, kita pasti dapat belajar untuk memiliki semua kombinasi elemen dari 5 kemungkinan yang ada.
  - Misalnya saat ini elemen yang diketahui S = (1, 2, 3, 5). Dapat dipelajari semua kombinasi elemen lain (1, 2, 3, 4), (1, 2, 3, 5), (1, 2, 4, 5), (1, 3, 4, 5), dan (2, 3, 4, 5) (Kecuali pada sesi bertarung pertama terpaksa menggunakan elemen awal). Maksimalkan jawaban yang didapat dari subsoal 3 untuk mengerjakan subsoal ini.

Solusi Subsoal 6

- Subsoal 6 (10 Poin) : $N \leq 1000$, $K \leq 10$
  - Perkenalkan bitwise dan bitmask, yang merupakan teknik menyimpan "*array of boolean*" dalam bilangan bulat.
  - Bisa juga dengan menjadikannya setiap digitnya menjadi skill yang bersesuaian. Misalnya 1234 berarti menguasai skill 1, 2, 3, dan 4.
  - Bisa juga dengan menyimpan setiap state sebagai dimensi DP
  - Lakukan dynamic programming.
  - Untuk setiap ruangnya, apabila perpustakaan maka skill nya dapat diubah salah satunya. Apabila ruang bertarung, maka harus dicek apakah skill set saat ini dapat mengalahkan ruangan bertarung saat ini.

Solusi Subsoal 7

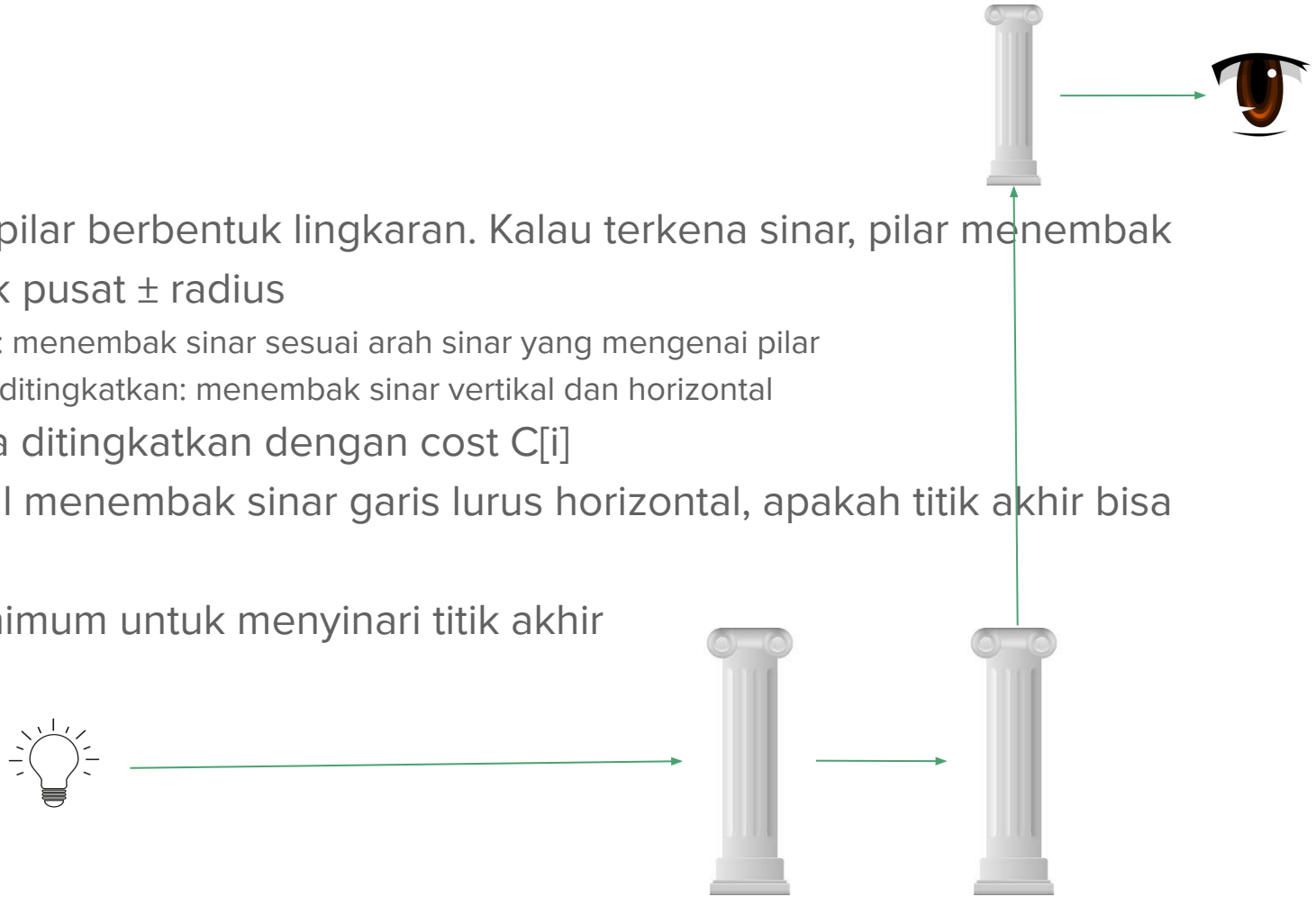
- Subsoal 7 (16 Poin) : $N \leq 1000$, $K \leq 20$
  - Perkenalkan bitwise dan bitmask, yang merupakan teknik menyimpan "*array of boolean*" dalam bilangan bulat.
  - Lakukan kompresi, perhatikan bahwa saat terurut, hanya akan ada C(20, 4) = 4845 kombinasi penyimpanan.
  - Sisanya, lakukan dynamic programming yang sama dengan subsoal 6.

Solusi Subsoal 8

- Subsoal 8 (15 Poin) : $N \leq 200'000$, $K \leq 10$
  - Perhatikan bahwa di antara setiap ruangannya, kita dapat melakukan kompresi untuk setiap sesi bertarungnya dengan teknik prefix sum.
  - Karena ruangan perpustakaannya hanya 100, maka dapat dilakukan dynamic programming yang mirip dengan subsoal 6.
- Subsoal 9 (10 Poin) : $N \leq 200'000$, $K \leq 20$
  - Gabungkan ide untuk Subsoal 7 dan 8

Pilar Ajaib

- Ada N buah pilar berbentuk lingkaran. Kalau terkena sinar, pilar menembak sinar dari titik pusat ± radius
  - Pilar biasa: menembak sinar sesuai arah sinar yang mengenai pilar
  - Pilar yang ditingkatkan: menembak sinar vertikal dan horizontal
- Pilar ke-i bisa ditingkatkan dengan cost C[i]
- Dari titik awal menembak sinar garis lurus horizontal, apakah titik akhir bisa tersinari
- Cari cost minimum untuk menyinari titik akhir



Solusi Tiap Subsoal

Subsoal 3 (N ≤ 1.000; C[i] = 0): **DFS E = N^2, O(N^2)**

Subsoal 4 (N ≤ 1.000): **Dijkstra E = N^2, O(N^2 log N)**

Subsoal 5 (-1000 ≤ x, y ≤ 1000; R[i] = 0; C[i] = 0): **DFS grid, O(MAX_X*MAX_Y)**

Subsoal 6 (R[i] = 0, C[i] = 0): **Point compression, DFS E = N, O(N log N)**

Subsoal 7 (C[i] = 0): **Grouping, DFS E = N, O(N log N)**

Subsoal 8 (Full): **Grouping, Update weight, Dijkstra E = N, O(N log N)**

Graph E = N^2 (C = 0)

- Tiap pilar jadi node
- Ada edge dari pilar i ke pilar j jika range X atau Y nya beririsan
- Solusi tinggal Connectivity Test (DFS)

Graph E = N^2

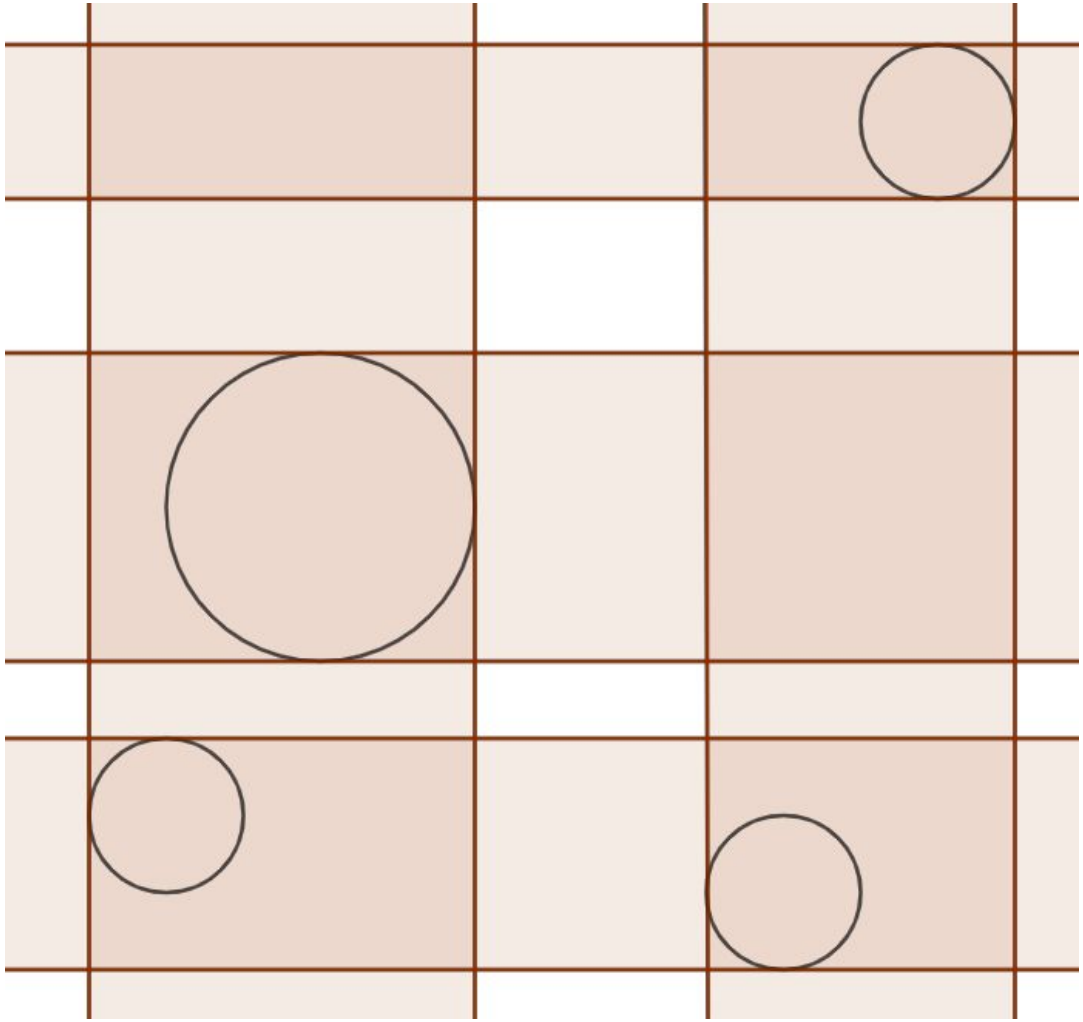
- Node nya ada 2N, masing-masing pilar bisa state vertikal/horizontal
- Graph nya sama kayak yang C = 0, bedain beririsan range X nya atau Y nya
- Untuk masing-masing pilar i, tambah edge yang bisa switch vertikal-horizontal dengan cost C[i]
- Solusi tinggal Dijkstra

Graph E = N (R = 0, C = 0)

- Buat yang X atau Y nya sama, di graph yang lama kan ada edge yang cost 0
  - Gabungin aja jadi 1 node besar
- Pilar jadi edge, bukan node
- Node nya distinct X + distinct Y
- Pilar ke-i menghubungkan node yang isinya X[i] dengan node yang isinya Y[i]
- Solusi tinggal Connectivity Test (DFS)

Graph E = N (C = 0)

- Mirip kayak yang R = 0
  - Bedanya bukan cuma X atau Y yang sama, tapi di grup X atau grup Y yang sama
- Node nya GrupX + GrupY
- Pilar ke-i menghubungkan node yang isinya GrupX[i] dengan node yang isinya GrupY[i]
- Solusi tinggal Connectivity Test (DFS)



Graph E = N

- Sama kayak C = 0, bedanya weighted
- Weight dari edge: minimum C[i] dari semua pilar yang ada di perpotongan GrupX[i] dan GrupY[i]
- Solusi tinggal Dijkstra

Mencari Bola



- Terdapat $N = 2^{10}$ kotak, dan tepat $K \leq 2$ kotak di antaranya terdapat bola
- Anda ingin tahu di mana bola berada dengan menanyakan beberapa pertanyaan
- Sekali bertanya, bisa nanya paling banyak 10 pertanyaan, tiap pertanyaan berisi subset dari {1, 2, ..., N}
- Tiap himpunan dijawab dengan YA apabila salah satu anggotanya adalah nomor kotak yang mengandung bola
- Cost dari sebuah pertanyaan adalah $1/\min(|S_i|)$, S adalah himpunan yang ditanya
- Minimalkan total cost

K = 1

- Cukup bertanya sekali
- Pada himpunan ke-i (0-based), tanyakan nomor-nomor kotak yang bit ke-i-nya nyala
  - Himpunan 0: {1, 3, 5, 7, ...}
  - Himpunan 1: {2, 3, 6, 7, 10, 11, ...}
  - dst
- Banyaknya anggota tiap himpunan tepat N/2 = 512
- Bit ke-i dari jawaban menyala jika dan hanya jika jawaban dari himpunan ke-i adalah YA
- Total cost: 2/N

K = 2 (79 poin)

- Mirip solusi K = 1
  - Pada pertanyaan pertama, tanyakan kotak dengan nomor yang bit ke-i nyala
  - Pada pertanyaan kedua, tanyakan kotak dengan nomor yang bit ke-i padam
- Misal a dan b adalah posisi bola. Jawaban pertanyaan di atas adalah:
  - ON = a | b
  - OFF = ~(a & b)
- Jika bit ke-i dari ON menyala tetapi OFF tidak, maka bit ke-i dari a dan b menyala
- Jika bit ke-i dari ON padam tetapi OFF tidak, maka bit ke-i dari a dan b padam
- Selain itu, maka tepat satu dari bit ke-i a dan b menyala. Kita anggap bit ini sebagai *unknown*

K = 2 (79 poin)

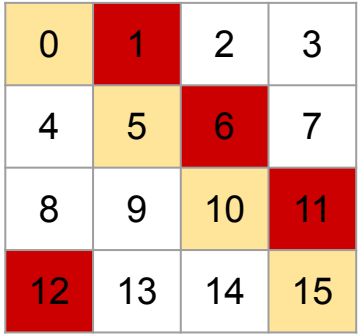
- Misalkan *mask* adalah bit-bit yang *unknown*.
  - a xor b = mask
- Tersisa paling banyak N/2 pasang kandidat jawaban yang mungkin
- Lakukan satu pertanyaan untuk N/2 kandidat ini
  - Lagi-lagi dengan menggunakan teknik serupa K = 1
- Total cost ≤ 6/N

K = 2 (menuju penuh) (oleh AMnu)

- Untuk seterusnya:
  - *Partisi menjadi X kelompok* adalah mempartisi kandidat-kandidat kotak yang masih mungkin berisi bola ke dalam X kelompok [0, 1, ..., X-1] yang isinya hampir sama banyaknya
  - *Menanyakan kelompok* $\{x_1, x_2, ..., x_k\}$ artinya membuat himpunan yang anggotanya merupakan anggota dari kelompok $x_1, x_2, ..., x_k$.
  - Setiap kali bertanya, kita selalu mengasumsikan kotak-kotak yang sudah tidak mungkin terdapat bola (bukan kandidat) selalu dimasukkan ke dalam himpunan
- Pada pertanyaan pertama, partisi menjadi 5 kelompok, tanyakan C(5, 3) = 10 kombinasi kelompok:
  - {0, 1, 2}, {0, 1, 3}, {0, 1, 4}, {0, 2, 3}, {0, 2, 4}, {0, 3, 4}, {1, 2, 3}, {1, 2, 4}, {1, 3, 4}, {2, 3, 4}
- Jawaban dari pertanyaan ini akan memberikan dua kemungkinan:
  - Kita tahu bahwa kedua bola berada di sebuah kelompok yang sama
  - Kita tahu bahwa kedua bola berada di dua kelompok yang berbeda

Kasus 1

- Pada pertanyaan pertama, kedua bola berada di kelompok yang sama
- Partisi menjadi 16 kelompok:
  - Tanyakan kelompok “baris” (4 himpunan)
  - Tanyakan kelompok “kolom” (4 himpunan)
  - Tanyakan kelompok kuning terang
  - Tanyakan kelompok merah gelap
- Dapat dibuktikan kita bisa mendapatkan salah satu:
  - Dua bola berada di kelompok berbeda
    - Untuk kasus ini, lakukan “solusi K = 1” pada kedua kelompok tersebut
  - Dua bola berada di kelompok sama
    - Untuk kasus ini, lakukan partisi 16 kelompok ini sekali lagi



Kasus 2

- Pada pertanyaan pertama, kedua bola berada di kelompok yang berbeda
- Partisi kandidat bola pertama menjadi 7 kelompok
- Partisi pula kandidat bola kedua menjadi 7 kelompok lainnya
- Untuk kandidat bola pertama, tanyakan kelompok:
  {0, 1, 2, 3}, {0, 1, 2, 3}, {0, 1, 2, 3}, {3, 4, 5, 6}, {3, 4, 5, 6}, {3, 4, 5, 6}, {0, 1, 5, 6}, {0, 1, 5, 6}, {1, 2, 4, 5}, {0, 2, 4, 6}
- Untuk kandidat bola kedua, tanyakan kelompok:
  {1, 3, 5}, {0, 3, 6}, {0, 1, 2}, {1, 3, 5}, {0, 3, 6}, {0, 1, 2}, {1, 3, 5}, {0, 3, 6}, {0, 1, 2}, {4, 5, 6}
- Kedua kumpulan himpunan tersebut ditanyakan secara “bersamaan”, sehingga hanya diperlukan satu pertanyaan
- Setelah itu, lakukan “solusi K = 1” di kandidat yang tersisa

K = 2

- Total cost dari kasus terburuk kedua kasus: < 4/N

Terima Kasih